GitHub Workload & Submission Plan

https://gemini.google.com/app/e909fe195b7c6269

User prompt: context: there are three team members who want equal distribution of the workload as reflected in the github commit history, the project is : "Goal Develop a machine learning solution to predict whether an online shopping session will result in a purchase. Your team will preprocess the data, develop and compare classification models, improve their performance, and use the final model to generate predictions. The challenge focuses on model development, validation, comparison, and improvement, with particular attention to evaluating classification models on an imbalanced dataset. Data (Data sourceLinks to an external site.) You will receive two datasets containing numerical and categorical variables that have already undergone basic cleaning: 1. Training dataset (challenge2_train.csvDownload challenge2_train.csv): - 9,247 observations including the target variable. - Use this dataset to develop and validate your models. - The target is Revenue. 2. Test dataset (challenge2_test.csvDownload challenge2_test.csv): - 3,083 observations without the target variable. - Use this dataset only to generate your final predictions. The ID variable uniquely identifies each observation. Do not use ID as a predictor. The target is imbalanced, so do not use accuracy alone to evaluate model performance. Tasks 1. Prepare the Data: Explore the data sufficiently to understand the prediction problem and prepare it for modeling. Apply appropriate preprocessing for the models. 2. Establish a Validation Strategy: Select an appropriate validation strategy and use it consistently to compare models, identify overfitting or underfitting, tune hyperparameters, and select your final model. Use F1 score for the positive (Revenue=True) class as the primary evaluation metric. Use additional classification metrics and the confusion matrix when appropriate to better understand model performance. 3. Develop and Compare Models: At minimum, develop and evaluate: - Logistic Regression - Naïve Bayes - K-Nearest Neighbors - Support Vector Machine - Decision Tree - Random Forest - Gradient Boosting You may use additional classification models. Compare the models using your validation results and summarize their performance in a table. 4. Improve Your Models: Use Grid Search to tune selected models. Feature engineering is encouraged when appropriate. 5. Select the Final Model: Select the model your team expects to generalize best to unseen data. Briefly justify your selection based on factors such as: - validation performance, - overfitting or underfitting, - model complexity, - computational requirements, - interpretability. Your final model may be one of the required models or an additional classification model investigated by your team. 6. Generate Predictions: Train your final model and generate predictions for challenge2_test.csv. Submit a predictions.csv file containing the ID and predicted Revenue class (True or False) for every observation in the test dataset. Competition Final predictions will be evaluated against the hidden test labels using F1 score for the positive (Revenue=True) class. Higher F1 score is better. Competition performance will contribute to the challenge grade, but the grade will also consider the quality of your modeling methodology and implementation. GitHub Submission Maintain your team's work in your GitHub repository. Your final submission must include: - final Jupyter notebook, - README.md, - predictions.csv, - any additional code required to run the notebook, - information necessary to reproduce your environment. The notebook should clearly show the complete workflow from loading the data to generating the final predictions. The README.md should briefly summarize the approach, validation strategy, models evaluated, final model, and final validation performance. All team members are expected to contribute meaningfully. GitHub activity will be used as evidence of participation. Each team member must submit a Peer-review form.docx Rules - Evaluate all seven required classification models. - You may include additional classification models and feature engineering. - Use only the training data for model development, validation, feature engineering, and hyperparameter tuning. - Use the test data only to generate final predictions. - Do not attempt to obtain or reconstruct the hidden test labels. - Do not use external

datasets unless authorized. - Do not share challenge-specific code, models, or predictions between teams. - Your final solution must be reproducible. The objective is to develop the best predictive model your team can produce while following sound machine learning practices." add nothing new or additional, keep the final submission to only final Jupyter notebook, README.md, predictions.csv,

Response: To achieve an equal workload distribution and a balanced GitHub contribution graph while restricting the repository strictly to the **final Jupyter notebook**, **`README.md`**, and **`predictions.csv`**, divide the project by both pipeline stage and model family.

Because a repository with only three files cannot rely on separate `.py` scripts or `requirements.txt` files, include the required environment reproduction details (Python version and library versions) directly inside a section of your `README.md`.

1. Workload & Notebook Section Allocation

Each team member owns specific sections of the single Jupyter notebook and specific sections of the `README.md`, ensuring comparable coding complexity, runtime responsibility, and documentation weight.

Team Member	Notebook Responsibilities	Models Owned	Deliverable & `README.md` Ownership
Member 1	**Task 1 (Data Prep):** Load `challenge2_train.csv` and `challenge2_test.csv`, drop `ID` from predictors, analyze `Revenue` class imbalance, perform EDA, and build the preprocessing pipeline (numerical scaling + categorical encoding). **Task 4 (Part 1):** Implement feature engineering transformations in the pipeline and tune assigned models via Grid Search.	• Logistic Regression • Naïve Bayes	• `README.md`: Project Approach, Data Preprocessing summary, and Environment Reproduction specs (Python & package versions).
Member 2	**Task 2 (Validation Strategy):** Set up Stratified K-Fold cross-validation and build the shared evaluation function calculating `Revenue=True` F1 score (primary), Precision, Recall, ROC-AUC, and Confusion Matrices. **Task 4 (Part 2):** Execute Grid Search hyperparameter tuning on assigned	• K-Nearest Neighbors • Support Vector Machine • Decision	• `README.md`: Validation Strategy and Baseline vs. Tuned Models Evaluated summary.

Team Member	Notebook Responsibilities	Models Owned	Deliverable & `README.md` Ownership
	models and diagnose overfitting/ underfitting.	Tree	
Member 3	**Task 3 & 4 (Part 3):** Train ensemble baselines, run Grid Search hyperparameter tuning on ensembles, and compile the unified 7-model validation comparison table.	• Random Forest	• `predictions.csv` (`ID` and predicted `Revenue` class for all 3,083 test rows).
	Task 5 & 6 (Selection & Predictions): Write the final model selection justification, retrain the winning model on the full training set, and export `predictions.csv`.	• Gradient Boosting	• `README.md`: Final Model Selection justification and Final Validation Performance.

2. Commit-by-Commit Execution Plan (7 Commits Per Member)

To prevent merge conflicts in a single `.ipynb` file and guarantee equal representation in the GitHub commit history, execute commits in a structured sequence. Each member makes **7 atomic commits** across the lifecycle of the project.

Phase 1: Skeleton, Data Prep & Validation Harness

1. **Member 1 — Commit 1:** Initialize the Jupyter notebook with Markdown headers for all 6 tasks and load the train/test datasets (`ID` separated).
2. **Member 1 — Commit 2:** Add EDA cells (missing value checks, feature distributions, and `Revenue` target imbalance breakdown).
3. **Member 1 — Commit 3:** Build the `scikit-learn` preprocessing `ColumnTransformer` (handling numerical and categorical variables using only training data).
4. **Member 2 — Commit 1:** Implement the Stratified K-Fold validation splitter and the standardized evaluation helper function (F1 score for `Revenue=True`, confusion matrix, and secondary classification metrics).
5. **Member 1 — Commit 4:** Initialize `README.md` with the environment reproduction specifications (exact versions of Python, `scikit-learn`, `pandas`, `numpy`) and the Data Preprocessing overview.

Phase 2: Baseline Model Development (Task 3)

6. **Member 1 — Commit 5:** Implement and validate baseline **Logistic Regression** and **Naïve**

Bayes models in the notebook.

7. **Member 2 — Commit 2:** Implement and validate baseline **K-Nearest Neighbors** and **Support Vector Machine** models in the notebook.
8. **Member 2 — Commit 3:** Implement and validate the baseline **Decision Tree** model in the notebook.
9. **Member 3 — Commit 1:** Implement and validate baseline **Random Forest** and **Gradient Boosting** models in the notebook.
10. **Member 3 — Commit 2:** Create the consolidated 7-model baseline comparison table in the notebook comparing validation F1 scores (`Revenue=True`), train vs. validation gaps, and confusion matrix summaries.

Phase 3: Feature Engineering & Grid Search Tuning (Task 4)

11. **Member 1 — Commit 6:** Add feature engineering transformations to the preprocessing pipeline and verify no data leakage occurs from the test set.
12. **Member 1 — Commit 7:** Run `GridSearchCV` (optimizing `f1` for `Revenue=True`) on Logistic Regression / Naïve Bayes and log results.
13. **Member 2 — Commit 4:** Run `GridSearchCV` on K-Nearest Neighbors and Decision Tree and log validation results.
14. **Member 2 — Commit 5:** Run `GridSearchCV` on Support Vector Machine and document overfitting/underfitting behavior.
15. **Member 3 — Commit 3:** Run `GridSearchCV` on **Random Forest** and log validation performance.
16. **Member 3 — Commit 4:** Run `GridSearchCV` on **Gradient Boosting** and update the master comparison table with tuned model scores.

Phase 4: Final Selection, Predictions & Documentation (Tasks 5 & 6)

17. **Member 3 — Commit 5:** Write the Task 5 Final Model Selection section in the notebook, evaluating validation F1 performance, overfitting/underfitting, model complexity, computational requirements, and interpretability.
18. **Member 3 — Commit 6:** Train the selected final model on the full `challenge2_train.csv` dataset, generate predictions on `challenge2_test.csv`, verify format (`ID`, `Revenue` as `True`/`False` for all 3,083 rows), and commit `predictions.csv`.
19. **Member 2 — Commit 6:** Update `README.md` with the Validation Strategy and the Models Evaluated performance table.
20. **Member 3 — Commit 7:** Update `README.md` with the Final Model Selection justification and Final Validation Performance metrics.

21. **Member 2 — Commit 7:** Perform a full top-to-bottom Restart & Run All on the Jupyter notebook to ensure 100% reproducibility and commit the final rendered `.ipynb` file.

3. Git Rules for Notebook Parity

- **Clear Cell Outputs During Development (Commits 1–20):** Jupyter notebooks store base64-encoded plots and HTML tables directly inside the `.ipynb` JSON file. If one member commits rendered charts while others commit code, GitHub's "lines added" metric will be heavily skewed. Clear cell outputs before committing during development, and only save the rendered outputs on Commit 21.
- **Pull Before Editing:** Because `.ipynb` files are structured as JSON, two members editing the notebook simultaneously will trigger merge conflicts. Use a sequential handoff (push to `main` or merge a Pull Request before the next member starts their notebook commit).
- **Final Repository Check:** Before final submission, verify the root directory contains strictly three tracked items (`git ls-files`):
    1. Your final `.ipynb` notebook
    2. `README.md`
    3. `predictions.csv`
